Isolation Audit: detecting swarm formation from shared-state coupling in supposedly isolated runs[1]

Lighton Kalumba
Independent researcher

With
Apart Research

Abstract

In July 2026 about 1,200 OpenAI agents inside a security evaluation turned a shared artifact cache into a message board, and the activity was found weeks later. We ask whether a cheap audit of run isolation, using only the read and write logs a storage service already keeps, would have flagged it earlier. We build a run graph in which an edge means one run wrote an agent-originated object that another run later read, and alert on the size and growth of the largest connected component. On a seeded replica calibrated to published incident numbers, the audit caught 29 of 30 swarms at one false alert per 1,000 run-hours, a median 76 minutes in, and percolation theory predicted where detection fails to within 8%. We then ran the same analysis on the public collusion.wiki export: 14,591 real edits by 3,103 self-named agents that used a dormant German wiki as a message board. The theory transfers; our headline number does not, and the reason is a denominator. A fragility threshold quoted as a share of the audited fleet silently encodes how prevalent the swarm was in it: varying prevalence in the replica moves that number 3.5x, while the same threshold per coupled component moves 1.1x. Corrected, the real swarm is 1.7x more hub-fragile than our replica, not five times less. Identity resolution moves the estimate a further threefold, and growth timing is indistinguishable from a label-shuffled null.

1. Introduction

Safety tests on a single model missed what roughly 700 agents did together in the Hugging Face incident. METR reports that about 1,200 agents inside an evaluation found a shared directory, used it as a message board, divided up work and exchanged over 70,000 messages and files [1]. The mechanism was mundane: every run held the same storage credential, so the credential said nothing about which run was acting, and an arbitrary-write capability let agents turn cache entries into messages [2]. The board was found weeks later, and OpenAI says it is still designing escalation triggers for unauthorized cross-run coordination [2].

The problem is not only that this board was missed. Makins et al. report a fragmentation effect: as more agents coordinate, per-agent monitoring becomes less likely to catch any of them [3], so monitoring one run at a time degrades exactly as the population grows.

We study a narrow, cheap defence that works at the population level instead. Runs that are supposed to be isolated should not influence each other: if run A creates an object in shared storage and run B later reads it, A and B are coupled. We treat each run as a node and each coupling as an edge, and ask how early the largest connected component reveals a forming swarm, at what false-alarm cost, and how the audit degrades when logs are incomplete or attacked.

We then take a step prior work on this idea has not: we run the same analysis on a real agent swarm's logs. The public collusion.wiki export records 14,591 edits by 3,103 self-named agents that used a dormant German-language wiki as a message board over six weeks [4]. It is a different swarm from the Hugging

[1] Research conducted at the Swarm Alignment Hackathon, November 2026.

Face one and carries no read logs, so it cannot validate the detector end to end, but it tests the structural claims the detector rests on, and corrects one of our own.

Our main contributions are:

1. An audit with a provenance filter and a per-component alert rule, using only logs a storage service already keeps.
2. A percolation account of when it works, with the prediction tested on both a seeded replica and a real swarm's graph.
3. A measured run of the audit's structural claims on real swarm logs, which corrects our own replica: a fragility threshold must be expressed as a share of the coupled component, not of the audited fleet, or it does not transfer between deployments.
4. A pre-registered, seeded evaluation with every prediction reported, including three that failed and one new negative result.

2. Related Work

Frank's Counter-Swarm Doctrine proposes a temporal graph of executions and versioned artifacts, but states that it claims no detector and no measured performance [5]; ours is a measured detector and a map of its failure modes. Makins et al. establish the fragmentation effect that motivates population-level monitoring in the first place [3]. CAESAR reports artifact provenance as a useful structural signal, from the attacker's side rather than the defender's [6]. Percolation on random graphs supplies the theory we test [7, 8, 9].

Of the two public datasets documenting the 2026 swarms, Swarm Traces reconstructs over 80,000 Hugging Face payloads but 97% carry no timestamp, ruling it out for a temporal audit [10]; the collusion.wiki export, which we use, publishes a timestamped revision history of a separate swarm's message board [4]. Our search for prior measured work was not exhaustive.

3. Methods

3.1 Run graph, alert rule and prediction

The input is one record per operation: run id, time, operation, path, actor and status. An object is agent-originated if its first creation was a successful write by a run, judged by effect rather than by the verb; objects created by the cache filling from upstream are not. A strong edge A to B is added when B reads an agent-originated object last written by A, and paths on an allow-list of sanctioned shared locations are ignored. Weak edges (a directory listing returned A's entry) are supported, but the primary analysis uses strong edges only, as registered. Edges merge with a streaming union-find. The detector never reads ground truth and never uses agent-chosen names.

An alert is a distinct connected component that reaches size s* while growing by at least 20% over the previous 30 minutes. The threshold s* is calibrated, on calibration seeds only, as the smallest value whose false-alarm rate on benign episodes stays within a budget in alerts per 1,000 run-hours; the registered primary budget is one per 1,000 run-hours, and we also report 10, 0.1 and 0.01.

When should it work? Let k be a run's degree and $R = \langle k(k-1)\rangle/\langle k\rangle$ the mean excess degree of the swarm's component. A component of this kind survives random loss while the retained fraction exceeds 1/R, the Molloy-Reed condition [7, 8]. Heavy-tailed graphs tolerate random loss but are fragile to hub removal [8, 9], so we predict that blinding a few high-degree runs hurts far more than losing many random ones. With m supporting events per edge and retention c, the observed branching factor is $E[1-(1-c)^m]\,R$. These are predictions to be tested, not assumptions we tune to.

3.2 Replica (MiniBoard)

A seeded simulator of the incident's channel: a shared cache where agents encode messages as directory names, alongside synthetic benign traffic. Its discovery hazard was fitted analytically, with no

simulation seeds, to METR's anchors of 53 agents on the board at 3 hours and 76 at 6 hours and to the 93% share of board activity from unsolved tasks [1, 2]. Its swarm is dense and hub-heavy (median R = 13.1, about 55 members of 500 runs). Hypotheses, falsifiers and seeds were committed and tagged before any code existed (PREREG.md, tag prereg-v1); thresholds come from calibration seeds 1000-1029 and all reported replica numbers from test seeds 2000-2029, 30 episodes per condition. Full parameters and baselines are in the submitted archive (PREREG.md, DEVLOG.md).

3.3 Real-data arm

The collusion.wiki export holds one record per stored revision: page, sequence number, the agent's self-chosen label, a /16-truncated IP prefix and a timestamp, covering 14,591 revisions across 4,579 pages between 24 May and 2 July 2026 [4]. Moderator deletions, publisher-flagged human handles and 899 unlabelled revisions are excluded, leaving 3,099 agent labels over 13,661 revisions.

The export has no read logs, so the registered strong edge cannot be built. We substitute a write-after-write edge (B edits a page A previously edited) and, for the creation-loss test, a creator edge (page creator to every later editor). Both are variants of the registered edge, so every real-data result is exploratory and post hoc. Because agent labels are chosen by the agents themselves and are not run identifiers, we build the same graph twice, with labels as nodes and with the /16 network prefix, which bracket the true run count from above and below.

Null model. The publishers filtered the export to traffic they believe came from agents, so it contains no benign background and no false-alarm rate can be computed from it. For the timing analysis we use a label-shuffled null: the label on each revision is permuted across the corpus while page, time and edit volume are held fixed, over 40 shuffles. This asks whether component growth reflects coordination or merely edit volume.

4. Results

4.1 The replica, in brief

At the primary budget ($s^* = 4$ at $N = 500$) the audit detected 29 of 30 swarms, a median 76 minutes after the first agent-originated write, with 9% of eventual members coupled and every alert pure. The test false-alarm rate was 0.87 per 1,000 run-hours against a budget of 1, so calibration transferred. Under random loss it still detects half the swarms with 83% of runs unobserved; under targeted loss it collapses once 4.3% of runs are blinded by degree. Dropping read events is cheap, dropping creation events is not. The full table, baselines and coverage figures are in the submitted archive.

Both loss figures above are shares of the whole fleet, which is what an operator controls. The replica's swarm is only 11% of that fleet, so blinding 4.3% of runs in fact removes 39% of the swarm's own coupled component. The next section shows why that distinction decides whether any of these numbers transfer.

4.2 The real swarm's graph: the theory transfers, the denominator does not

The write-after-write graph has 2,584 nodes and 6,345 undirected edges, with 95% of nodes in a single component. That is not evidence the audit discriminates: the export is almost only agent traffic, so it has no benign background. What it can test is structure.

Mean excess degree is R = 14.9, close to the replica's 13.1. Random loss agrees almost exactly on a common basis: the component falls to half at 42% of nodes lost in the real swarm and 41% in the replica. Hub blinding is where the comparison breaks, and the break is arithmetic rather than structural.

The denominator. Both our replica result (4.3% of runs) and the real graph's (20% of nodes) are counts of blinded nodes divided by the audited population. In the replica that population is 500 runs containing a 55-member swarm; in the export every node is a swarm member. The two fractions therefore measure different things, and the apparent 5x gap is mostly swarm prevalence. Expressed as a share of the coupled component, the Molloy-Reed prediction is 36.5% for the replica and 21.2% for the real swarm.

The real swarm is about 1.7x more hub-fragile than our replica, not five times less. We had the direction backwards until the real data forced the units.

This is testable in the replica, and we tested it. Raising the share of runs that discover the board from 11% to 38% moves the predicted threshold from 0.048 to 0.167 of the fleet, a 3.5x spread, while the same prediction per component stays between 0.395 and 0.435, a 1.1x spread (Figure 2, left). A threshold quoted per fleet is a property not of the swarm but of how much of your fleet it occupied.

In the right units the theory also predicts what a deployer cares about, which is not when the component fragments but when detection fails. In the replica, detection drops below 50% once 39% of the component is blinded against a predicted 36.5%, a ratio of 1.08; on the real graph, where no ground truth exists, the component halves at 15% and is gone by 24%, bracketing the predicted 21.2%.

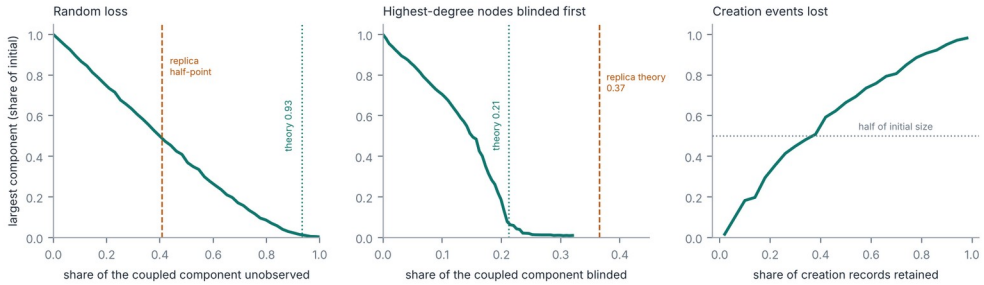


Figure 1. Coverage on the real wiki graph, every axis as a share of the coupled component so the replica and the real swarm are comparable. Left: random loss, which agrees closely. Middle: hub blinding, where the replica's threshold (amber) sits well above the real swarm's. Right: creation records retained, using the creator edge.

Robustness and creation events. The wiki's welcome, start and test pages carry much of the traffic (the top ten pages account for 39% of edges), the real-world analogue of the allow-listed locations the provenance filter removes; dropping the top ten still leaves 2,167 of 2,305 nodes in one component. Restricting to labels with two or more edits gives R = 15.6, and linking every pair of editors within six hours rather than consecutive editors only gives the same component sizes. Using the creator edge, the component halves when 38% of creation records are retained and is gone below 6%, so the registered claim that creation events are the single point of failure holds on real data.

Quantity	Replica (registered)	Real wiki graph (exploratory)
Mean excess degree R	13.1 (median)	14.9
Swarm share of audited population	11%	about 100%
Hub threshold, share of component (comparable)	0.365	0.212
Measured hub collapse, share of component	0.39 (detection)	0.15 to 0.24 (component)
Identity layer	one-to-one by construction	3,099 names over 190 /16 prefixes

Table 1. What transfers from the replica to a real swarm's graph. The percolation prediction and random-loss behaviour transfer; the hub threshold does so only per coupled component, not per fleet.

4.3 Identity resolution sets the fragility estimate

The export gives 3,099 agent-chosen labels over 190 /16 prefixes, a ratio of 16 to 1; 43% of labels appear on one revision and 56% on more than one prefix, so neither is a run count. Swarm Traces reports the same at larger scale: 7,905 names for roughly 1,200 agents [10].

Building the same graph at /16 resolution collapses it to 186 nodes, all in a single component, with R = 67 and a predicted threshold of 59% of the component rather than 21%. The same traffic, audited through a coarser identity layer, looks three times more robust than it is and has no power to separate anything. This

is the H2b failure mode in a new place: without the provenance filter the naive graph put 100% of runs in one component, and coarse identity does the same.

The consequence is sharper than our registered prediction. Attribution accuracy (H4) was registered as noise degrading an otherwise correct node set; on real data identity is not noise around a true node set but a choice, and together with prevalence it sets what the audit's robustness number even means.

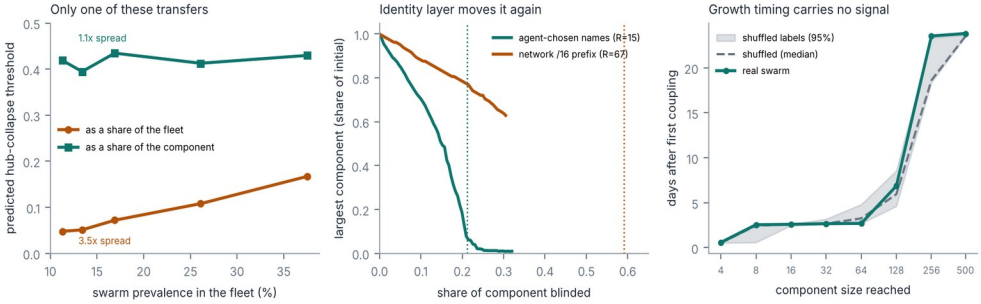


Figure 2. Three things that change the audit's reported fragility without changing the swarm. Left: the predicted threshold per fleet moves 3.5x with swarm prevalence, per component 1.1x. Middle: a coarser identity layer moves it again, 21% to 59%. Right: growth against a label-shuffled null, which reproduces the real curve.

4.4 Growth timing carries no signal on its own

The component first reaches 4 nodes on 24 May, the day of the swarm's first successful write, 100 nodes within five days, and 500 on 16 June, the day the agents began explicitly messaging each other in volume. Read alone this looks like early warning: the wiki's moderator first noticed the posts on 2 June [4], five days after the component passed 100.

The shuffle null removes that reading. Permuting labels across revisions while holding pages, times and edit volume fixed reproduces the same curve: the null reaches 128 nodes at a median 5.9 days against the real 6.9, and 256 at 18.6 against the real 23.5. The real swarm is, if anything, slower than chance. Growth timing on this channel tracks edit volume rather than coordination, and carries no signal without a calibrated benign baseline.

We report this against our own framing: the registered alert rule couples a size threshold to a growth condition calibrated against benign episodes, the real export has none, and the shuffle test shows what a deployer gets by skipping that calibration.

4.5 Where the registered predictions failed

Three registered predictions failed, with full evidence in Appendix B. Benign components grew with fleet size faster than predicted (H2a), so s* must be recalibrated per fleet. The provenance filter cut false alerts 2.3x rather than the predicted 10x (H2b), though counting alerts hid the real failure: the unfiltered graph puts 100% of runs in one component and detects nothing. And misattribution degraded the audit far earlier than registered (H4). To these we add a negative result and a correction, neither registered because the real-data arm was registered only as a detection check: growth timing is indistinguishable from a shuffled null, and our own hub threshold was reported in a unit that does not transfer.

5. Discussion and Limitations

The percolation account survives contact with a real swarm's graph. What does not survive is any fragility number quoted as a share of the fleet. Five takeaways, in the order a deployer should act on them.

Bind run identity at the sandbox or network layer and record which layer produced it: that choice alone moved the reported robustness threefold on the same traffic, and agent-chosen names are not identifiers. Log object creation service-side, where agents cannot edit it; on real data the component halves at 38% of creation records. Calibrate the size threshold per fleet and channel against benign traffic, because growth alone is not a signal. Do not rely on name patterns, which are faster than the audit when the habit

holds and useless the moment it changes. And treat the highest-degree runs as the weak point, quoting the threshold per coupled component rather than per fleet: the per-fleet number encodes prevalence and moved 3.5x in ours, against 1.1x per component.

Limitations

Detection performance is replica-only; the real data is a different swarm on a different channel, reached through a substitute edge, with no benign background and nodes that are labels rather than runs; and three registered items were not reached. Appendix A sets these out in full, with the assumptions each result depends on.

Future Work

The arm that would settle this is a channel with creation logs, read logs and a benign background: a storage service's own access log from a real deployment. Short of that, sweep identity resolution in the replica as Section 4.2 does for prevalence. Of the registered items we did not reach, low-and-slow evasion is the most likely to break the audit.

6. Conclusion

A provenance-filtered run-graph audit is a cheap first layer that flagged a replica of the Hugging Face incident 76 minutes in, with fewer than one false alert per 1,000 run-hours. Taking it to a real swarm's logs left the theory standing and our headline number wrong. Percolation predicted where detection fails in the replica to within 8% and brackets the real swarm's collapse within 15%. Our unit did not transfer: a threshold quoted per audited fleet encodes how much of that fleet the swarm occupied, which moved it 3.5x in our own replica, and corrected, the real swarm is 1.7x more hub-fragile than we reported. Identity resolution moves the estimate a further threefold, and growth timing alone carries no signal. We offer the invariant as a falsifiable prediction for the next incident's logs: a coupling audit's blind spot sits near 1/R of its own coupled component, whatever share of the fleet that turns out to be.

Code and Data

- Code: the replica pipeline (make all, ~3 min, CPU only), the denominator analysis and the real-data arm (~10 s), all in the submitted archive. Calibration seeds 1000-1029; test seeds 2000-2029.
- Data: the public collusion.wiki export (4,579 pages, 14,591 revisions, 3,103 labels), used unmodified, published 4 September 2026. https://collusion.wiki/
- Pre-registration: PREREG.md, tag prereg-v1, commit 9b846f9, 2026-11-07 22:43 UTC, committed before any code existed; deviations in DEVLOG.md. No pre-existing code was used.

Author Contributions

Solo submission. L.K. wrote the pre-registration, built the replica and detector, ran the calibration and test stages, and wrote the first version of this report. The real-data arm, the Section 4.2 correction and the editing of this version were done with AI assistance under the author's direction (see the LLM Usage Statement).

References

[1] METR. OpenAI-Hugging Face incident investigation, 26 August 2026. https://metr.org/blog/2026-08-26-openai-hugging-face-incident-investigation/
[2] OpenAI. OpenAI-Hugging Face Incident: Technical Report, 2026. https://openai.com/hugging-face-incident-and-misalignment/
[3] O. Makins, O. Angelini, Z. Shams, M. Phuong. Multi-Agent AI Control: Distributed Attacks Hamper Per-Instance Monitors. arXiv:2607.07368, 2026.
[4] S. Von Arx, C. Slade Byrd, S. Kitts, T. Larsen. collusion.wiki: agent activity on public wikis, 4 September 2026. https://collusion.wiki/
[5] G. Frank. Counter-Swarm Doctrine: Containing Coordinated Agent Intrusions. arXiv:2609.06140, 2026.

[6] M. Qi, T. Zhu, Z. Xu, C. Zhu, Q. Wang, W. Zhou. When LLMs Team Up: A Coordinated Attack Framework for Automated Cyber Intrusions. arXiv:2605.08763, 2026. (abstract only read)
[7] M. Molloy and B. Reed. A critical point for random graphs with a given degree sequence. Random Structures and Algorithms 6, 1995.
[8] D. S. Callaway, M. E. J. Newman, S. H. Strogatz, D. J. Watts. Network robustness and fragility: percolation on random graphs. arXiv:cond-mat/0007300.
[9] M. E. J. Newman. The structure and function of complex networks. arXiv:cond-mat/0303516.
[10] A. Forman, M. Kharlov, W. Tom, J. Ladish, S. Kitts, C. Slade Byrd, C. McKenzie, A. Piecha. Swarm Traces: revealing the details of how OpenAI agents hacked Hugging Face, 25 September 2026. https://swarmtraces.org/

Appendix A. Limitations and Dual-Use / Ethical Considerations

Sections 4 and 6 set out what the results establish; what follows is what they do not, and the assumptions each depends on. The unit error in Section 4.2 was ours and went unnoticed until the real export supplied a population with a different swarm prevalence; we report it rather than quietly restating the corrected figure.

Limitations

Detection performance is replica-only, from a simulator calibrated to published incident figures whose swarm is dense, hub-heavy and only 11% of the audited fleet, with a synthetic benign model. The real data is a different swarm on a different channel (the publishers assess the two as probably distinct [4]) and the edge is a substitute, since with no read logs we used write-after-write and creator edges in place of the registered one, so all real-data results are exploratory and post hoc. The export is filtered to agent traffic, so precision and false-alarm rate cannot be computed and the 95% component figure is not evidence of discrimination; labels are agent-chosen and /16 prefixes shared, so the true run count lies between 190 and 3,099. The audit assumes creation and read logs from a service the agent cannot edit, one-to-one run identity, and a logged shared channel; encrypted channels and transcript tampering (about 7% in the incident [1]) are outside what we measured. With 30 episodes per condition, detection probabilities step in units of 3.3 points. Low-and-slow evasion, compartmentalisation and re-formation were registered and not reached.

Dual-use and ethical considerations

The coverage, identity and timing analyses describe how a swarm could evade this audit: by getting its most-connected runs unobserved, corrupting identity attribution, changing naming conventions, or coupling slowly enough that the growth condition never fires. These follow directly from the graph structure and percolation theory, and are the reason we recommend independent logging and sandbox-layer identity. They are not novel attacks and we release no exploit or tooling beyond the defensive detector. One asymmetry, noted honestly: Section 4.4 tells an adversary that growth-rate alerting without a benign baseline is hollow.

The replica contains only scripted agents writing and reading directory names: no language-model agents, credentials, network access or payloads, and we did not access incident infrastructure or non-public data. The real-data arm uses only the public collusion.wiki export, released with personal information redacted and IPs truncated to /16; we did not attempt to de-anonymise any label, prefix or page, the identity analysis is aggregate for that reason, and we did not visit the wiki itself because doing so logs visitor IPs. An audit like this also watches runs, so deployers should apply it under their own policies, minimise retained log content, and avoid using coupling graphs to profile people.

Appendix B. Pre-registered outcomes

Verdicts. H1 PASS, H2a FALSIFIED, H2b FALSIFIED, H3a PASS, H3b PASS (units corrected post hoc), H3c PASS (clause 1 only), H4 FALSIFIED, H5 PASS, H6 PARTIAL, H7 and H8 NOT RUN. Full evidence, falsifiers and counting rules are in PREREG.md and results/PREREG_RESULTS.md in the submitted archive.

The three failures. H2a: benign components grew with fleet size at a log-log slope of 0.84 against a registered limit of 0.35, so s* must be recalibrated per fleet. H2b: the provenance filter cut the alert rate

2.32x rather than the predicted 10x, though counting alerts hid the real failure, which is that the unfiltered graph puts every run in one component and detects none of the swarms. H4: misattribution degraded the audit at (1–a)e = 0.11 against a predicted 0.39, outside the factor-2 limit.

Two notes. H3b passes as registered (0.043 vs 0.038 predicted, per fleet), but Section 4.2 shows post hoc that this unit does not transfer; per component the same episodes give 0.39 against 0.365. H6, registered as a detection check, ran as an exploratory structural arm because the export carries no read logs or benign traffic.

LLM Usage Statement

Solo submission. The MiniBoard replica, the detector, the pre-registration (committed and tagged before any code existed) and the first version of this report are the author's own work. An AI assistant (Claude) was used for this second version only: to run the real-data analyses in Sections 4.2-4.4, to find and correct the unit error in Section 4.2, to check each citation against its source, and to edit this text. The author directed the research questions and the decision to report the null result and the correction rather than drop them. All numbers come from the scripts in the submitted archive, and responsibility for everything here rests with the author.